PlainSpeak

Technical Documentation

VERSION	MVP - latest commit 7d508f8
STACK	Python 3.12 FastAPI Anthropic Claude Reson8 pypdf uv
NOTE	Synthetic-data demo only | all patient data is fictional

SECTIONS IN THIS DOCUMENT

1. System Architecture
2. Data Model
3. Storage Layer
4. HTTP API
5. PDF Extraction
6. Transcript Import
7. AI Subsystem
8. Voice / Speech
9. Frontend Architecture
10. Workflow State Machine
11. Demo Fixtures
12. Configuration
13. Testing
14. Security Notes

1. System Architecture

PlainSpeak is a single-process FastAPI application. There is no database -- each case is one JSON file on disk. The browser is a plain HTML/CSS/JS frontend served by the same process. There are no build steps, bundlers or external queues.

```
Browser (static HTML/JS)
       |
       | HTTP + WebSocket
       v
FastAPI (main.py)
  |-- api.py          - HTTP routes, workflow gates
  |-- storage.py      - atomic JSON case files
  |-- voice.py        - Reson8 token issuance
  |-- pdf_extract.py  - pypdf -> passages, lines, patient name
  |-- transcript_import.py  - text transcript -> speaker-labelled segments
  `-- ai/
       |-- client.py  - Anthropic SDK wrapper
       |-- prompts.py - system prompts + JSON schemas
       |-- summary.py - draft generation (one Claude call)
       |-- agent.py   - post-care agent loop (tool-use)
       `-- tools.py   - tool handlers (get_case_context, propose_progress_update)
```

The browser never holds permanent API keys. The Reson8 API key lives only on the backend; the browser receives a short-lived token from POST /api/voice/token that grants a 10-minute WebSocket connection directly to the Reson8 Realtime endpoint.

2. Data Model (contracts.py)

All models derive from Contract(BaseModel) with alias_generator=to_camel, so field names are snake_case in Python and camelCase over the wire (HTTP JSON and the case JSON file).

2.1 Core Enums / Literals

Type	Values
WorkflowState	awaiting_review, approved, post_care_active
SpeakerRole	clinician, patient, unknown
ResponsibleParty	patient, caregiver, care_team, unspecified
ReplyKind	answer, clarification, unresolved, update_confirmation
ReportedStatus	done, in_progress, blocked
TaskStatus	open, done, in_progress, blocked

2.2 Case -- the root entity

```
class Case(Contract):
    case_id: str
    patient_name: str | None = None   # NEW: extracted from discharge PDF on upload
    workflow_state: WorkflowState = "awaiting_review"
    sources: list[Source]             # discharge docs + consultation source
    draft_summary: DraftSummary | None
    summary_versions: list[SummaryVersion]  # immutable approved snapshots
    approved_summary_version: int | None
    care_tasks: list[CareTask]
    messages: list[Message]           # caregiver <-> assistant chat
    pending_updates: list[PendingUpdate]
    progress_events: list[ProgressEvent]
    unresolved_questions: list[UnresolvedQuestion]
    transcript_segments: list[TranscriptSegment]
    speaker_roles: dict[str, str]     # key = "sessionId:speakerId"
    input_revision: int               # incremented on every source change
```

Key invariants:

- input_revision is incremented every time sources or the transcript change. An approval or draft is only valid if its stored input_revision matches the case's current value.
- approved_summary_version points into summary_versions. Once a version is saved it is never mutated -- subsequent approvals append new versions.
- post_care_ready() returns True only when approved_summary_version is set AND approved.input_revision == case.input_revision -- i.e. no source changes since approval.
- patient_name is set once (on first document upload) and kept for all subsequent renders.

2.3 Source and Passage

```
class Source(Contract):
    id: str                           # "discharge_001" or "consultation_001"
    filename: str
    uploaded_at: str                  # ISO-8601 UTC
    passages: list[Passage]
```

```
    kind: Literal["document", "consultation"] = "document"

class Passage(Contract):
    id: str                             # "page_1_p3" or segment id
    page: int                           # 0 for consultation passages
    text: str
```

The consultation source (id="consultation_001") is rebuilt from transcript_segments and speaker_roles whenever either changes. Each passage text is prefixed with the speaker label: [clinician at 4000ms] Good morning Alex.

2.4 TranscriptSegment

```
class TranscriptSegment(Contract):
    id: str          # "seg_<uuid>" (live) or "seg_<session>_<n>" (imported)
    session_id: str  # UUID (live) | "manual_<uuid>" | "pdf_<sha256_prefix>"
    text: str        # max 10,000 chars
    speaker_id: int | None   # 0-7
    start_ms: int | None
    duration_ms: int | None
    recorded_at: str
```

Speaker keys (sessionId:speakerId) link segments to entries in speaker_roles. A role already confirmed by the clinician is never overwritten by a later PDF import.

2.5 SummaryVersion -- immutable snapshot

```
class SummaryVersion(Contract):
    version: int
    summary: str
    care_tasks: list[CareTask]
    source_ids: list[str]
    approved_by: str
    approved_at: str
    input_revision: int
    sources_snapshot: list[Source]   # deep copy of all passages at approval time
```

The sources_snapshot is taken at the moment of approval. Post-care answers are grounded against this snapshot, not live sources, so the evidence base is frozen even if documents are later replaced.

2.6 CareTask

```
class CareTask(Contract):
    id: str               # "task_001", "task_002", ...
    instruction: str
    timing: str | None
    responsible_party: ResponsibleParty
    source_refs: list[SourceRef]   # each = { sourceId, passageId }
```

Every task must cite the passages it was derived from. The API validates all passageId values exist in case.passage_index() before approving.

2.7 Progress tracking

```
caregiver reports -> PendingUpdate (status="pending")
    v clinician/caregiver confirms
ProgressEvent (immutable)
```

```
class PendingUpdate(Contract):
    id: str
    task_id: str
    reported_status: ReportedStatus
    note: str
    status: Literal["pending", "confirmed", "rejected"]
    approved_summary_version: int
    created_at: str
    resolved_at: str | None

class ProgressEvent(Contract):
    id: str
    task_id: str
    status: ReportedStatus
    note: str
    actor: str
    timestamp: str
    pending_update_id: str
    approved_summary_version: int
```

task_status(task_id) scans progress_events for the current approved_summary_version and returns the last event's status (defaults to "open").

3. Storage Layer (storage.py)

- One JSON file per case: data/cases/<case_id>.json
- Atomic writes: write to a temp file in the same directory, then os.replace(). A crash mid-write leaves the previous version intact.
- Per-case lock: a threading.Lock in the in-memory CaseStore instance prevents concurrent writes to the same case within the same process.
- store.update(case_id) is a context manager that loads, yields the Case object, and writes it back atomically on clean exit.

```
with store.update(case_id) as case:
    case.workflow_state = "approved"
# file is written here
```

4. HTTP API (api.py)

Base path: /api. All request/response bodies are JSON (camelCase). Errors are standard FastAPI HTTPException with a plain-text detail string.

4.1 Case lifecycle

Method	Path	Purpose
POST	/api/cases	Create (or reset) the demo case
GET	/api/cases/{case_id}	Load case + projected checklist

Response: CaseView -- { case: Case, checklist: TaskView[] } where each TaskView adds status and lastEvent to the base CareTask.

4.2 Document upload

```
POST /api/cases/{case_id}/document
Content-Type: multipart/form-data  |  Body: file (PDF, max 10 MB)
```

- Validates PDF magic bytes (%PDF).
- Extracts passages via extract_passages() (stable IDs: page_{n}_p{k}).
- NEW: Extracts patient name via extract_patient_name() and stores it on case.patient_name.
- If same filename and passages already on the case, returns immediately (no-op).
- Replaces the discharge_001 source, increments input_revision, resets draft_summary, sets workflow_state = "awaiting_review".

4.3 Transcript management

Live / manual save:

```
POST /api/cases/{case_id}/transcript
Body: { segments: TranscriptSegment[], speakerRoles: { [key]: role } }
```

Segments are appended; existing IDs are idempotent retries. A saved segment with a different body returns HTTP 409.

PDF transcript import:

```
POST /api/cases/{case_id}/transcript/document
Content-Type: multipart/form-data  |  Body: file (PDF)
```

- Content-addressed session ID: pdf_<sha256[:16]> -- re-uploading the same file is a no-op.
- Extracts lines via extract_lines(), parses via parse_transcript().
- Calls _apply_transcript(..., replace=True) -- imported transcript replaces prior capture.
- Roles already confirmed by the clinician are never overwritten by label inference.

Clear transcript:

```
POST /api/cases/{case_id}/transcript/clear
```

Blocked after any approval (approved_summary_version is not None -> HTTP 409).

4.4 AI workflow

```
POST /api/cases/{case_id}/summary  -> triggers generate_draft(), sets draft_summary
```

```
POST /api/cases/{case_id}/approve  -> body: ApproveRequest -> creates SummaryVersion
POST /api/cases/{case_id}/chat     -> body: { text } -> runs agent loop -> ChatResponse
```

/approve validates: 1. Summary and at least one task must be non-empty. 2. All sourceRefs must exist in case.passage_index(). 3. If inputRevision is sent, it must match the current case.input_revision.

4.5 Progress updates

```
POST /api/cases/{case_id}/updates/{update_id}/confirm
POST /api/cases/{case_id}/updates/{update_id}/reject
```

4.6 Voice token

```
POST /api/voice/token
Response: { accessToken, realtimeUrl, expiresAt }
```

The backend exchanges the permanent RESON8_API_KEY for a short-lived token. The browser opens the WebSocket directly to Reson8's realtimeUrl using the token in the Sec-WebSocket-Protocol header (bearer, <token>).

5. PDF Extraction (pdf_extract.py)

Built on pypdf. Three extraction functions:

5.1 extract_passages()

```
extract_passages(pdf_path: Path) -> list[Passage]
```

Splits page text on blank lines. Each chunk becomes one passage with a stable ID: page_{n}_p{k} (1-indexed page, 0-indexed chunk on that page). Max 200 passages total.

5.2 extract_lines()

```
extract_lines(pdf_path: Path) -> list[str]
```

Returns raw lines from all pages in order. Used by transcript_import.parse_transcript().

5.3 extract_patient_name() -- NEW

```
extract_patient_name(pdf_path: Path) -> str | None
find_patient_name(lines: list[str]) -> str | None
```

Scans every line for a patient name using a two-pattern regex:

```
_PATIENT_NAME = re.compile(
    r"(?:\bPatient(?:\s+name)?\s*:|^\s*Patient(?:\s+name)?\s{2,})\s*"
    r"([^\s(),.;:][^(),.;:]*?)\s*(?=[(),.;]|\s{2,}|\s\w+\s*:|$)",
    re.IGNORECASE,
)
```

Handles both labelled layouts ("Patient: Alex Example") and wide-gap table layouts ("Patient Anna Mueller Discharge date 24 October 2026"). Returns the first match, or None if not found. All three functions raise NoTextError if pypdf extracts no text (image-only or encrypted PDFs).

6. Transcript Import (transcript_import.py)

Parses a written consultation transcript into TranscriptSegment objects.

6.1 Expected format

```
[00:00:04] Clinician: Good morning Alex.       <- turn 1, startMs=4000
Patient: I thought I only needed my ID card.   <- turn 2, no timestamp
and may have misremembered the date.           <- continues turn 2 (no label)
```

Rules:

- Each new turn starts with a speaker label (1-4 words) followed by colon.
- An optional timestamp [HH:MM:SS], (MM:SS) or MM:SS.mmm precedes the label.
- A line with no label is appended to the previous turn (handles PDF line-wrapping).
- Lines before the first labelled turn are discarded (headers, titles).

6.2 Header block detection -- NEW

Many exported transcript PDFs open with a metadata block before the first spoken turn:

```
Patient: Anna Mueller
Clinician: Dr. Weber
Date: 24 October 2026
Purpose: Post-operative review
```

Previously these lines would be parsed as speaker turns (e.g. "Patient" appearing to say "Anna Mueller"). The new _header_length() function detects and skips this block.

Detection logic:

- A _METADATA_LABELS frozenset identifies document-field labels: date, time, purpose, note, location, participants, consent, and 20+ others.
- The block continues as long as lines are either metadata fields or short participant lines (up to HEADER_FIELD_WORDS = 6 words of text).
- _is_metadata() also filters stray metadata lines that appear after the header.

```
_METADATA_LABELS = frozenset({
    "date", "time", "duration", "purpose", "note", "notes", "title", "subject",
    "location", "setting", "language", "reference", "case", "case id", "encounter",
    "hospital", "clinic", "department", "ward", "session", "visit", "recorded",
    "recording", "transcript", "transcribed by", "participants", "consent",
})
```

6.3 Role inference

Label words (case-insensitive)	Inferred role
clinician, doctor, dr, physician, consultant, nurse, gp, arzt, arztin	clinician
patient, patientin	patient
Anything else (e.g. Speaker 1)	unknown

6.4 Validation limits

Constraint	Limit
Max turns	300
Max distinct speakers	8
Max chars per turn	10,000
Min distinct speakers	2
Max label words	4

6.5 Segment ID scheme

```
id = f"seg_{session_id}_{index:03d}"   # e.g. "seg_pdf_a3b7c9f1d2e4_001"
```

Session ID is content-addressed (pdf_<sha256[:16]>), so segment IDs are stable across re-uploads of the same file.

7. AI Subsystem (ai/)

7.1 Claude client (client.py)

Thin wrapper over anthropic.Anthropic. One core method:

```
def call(
    system: str,
    messages: list[dict],
    tools: list[dict] | None = None,
    schema: dict | None = None,
) -> anthropic.Message
```

When schema is supplied, the call uses tool_choice={"type": "tool"} with a synthetic structured_output tool -- structured JSON output without exposed tool semantics. Raises AIError (subclasses: AIAuthError, AIRateLimitError) on failures. Model and effort are read from env vars ANTHROPIC_MODEL and ANTHROPIC_EFFORT.

7.2 Summary generation (summary.py)

Single Claude call. Documents are formatted as XML:

```
<document sourceId="discharge_001" kind="document">
  <passage passageId="page_1_p0">...</passage>
</document>
<document sourceId="consultation_001" kind="consultation">
  <passage passageId="seg_...">...</passage>
</document>
```

Claude returns SUMMARY_SCHEMA JSON:

```
{
  "summary": "...",
  "careTasks": [
    {
      "instruction": "...",
      "timing": "...",
      "responsibleParty": "patient|caregiver|care_team|unspecified",
      "sourceRefs": [{ "sourceId": "...", "passageId": "..." }]
    }
  ],
  "missingInformation": ["..."]
}
```

All passageId values are validated against the known passage index after parsing. Tasks with unverifiable references are silently dropped before returning DraftSummary.

7.3 Post-care agent loop (agent.py)

Multi-turn tool-use loop with a fixed iteration cap. Claude may call:

Tool	When triggered
get_case_context	Before answering any case-specific question

propose_progress_update	When caregiver reports progress on a task

The loop exits when Claude returns a structured AGENT_REPLY_SCHEMA response:

```
{
  "kind": "answer|clarification|unresolved|update_confirmation",
  "text": "...",
  "sourceRefs": [{ "sourceId": "...", "passageId": "..." }],
  "eventRefs": ["evt_..."],
  "pendingUpdateId": "upd_...|null"
}
```

All sourceRefs are validated against the approved snapshot before reaching the frontend.

7.4 Tool handlers (tools.py)

get_case_context(case_id) Returns: approved summary, tasks with status, all source passages from the approved snapshot, last 10 messages, progress events for the current version, open questions.

propose_progress_update(case_id, task_id, reported_status, note) Creates a PendingUpdate (status "pending") and returns its pendingUpdateId. The update is not applied until the caregiver confirms via POST /updates/{id}/confirm.

record_progress_update(store, case_id, update_id, actor) Moves the pending update to "confirmed" and appends an immutable ProgressEvent.

reject_progress_update(store, case_id, update_id) Sets status to "rejected". No event appended.

7.5 System prompt design

Summary prompt constraints:

- "Only restate instructions that appear in the passages. Never add, infer or change clinical instructions, doses, dates or timings."
- "Patient statements are not instructions. An unconfirmed speaker is not a clinician."
- "Flag any disagreement between the transcript and the documents in missingInformation."

Chat prompt constraints:

- "Before answering anything about this case, call get_case_context."
- "If the approved material does not establish an answer, say so plainly, do not guess, and use kind 'unresolved'."
- "The update is not saved until the caregiver confirms in the app; never claim it is saved."

8. Voice / Speech Integration

8.1 Consultation capture (live, consultation.js)

The browser opens a WebSocket to Reson8 Realtime with these parameters:

```
encoding=pcm_s16le  sample_rate=16000  channels=1
language=<selected>  diarize=true  max_speakers=2
include_interim=true  include_timestamps=true
```

Audio is captured by an AudioWorkletNode (pcm16-downsampler) which downsamples to 16 kHz mono PCM and streams raw ArrayBuffer frames over the WebSocket.

Message types from Reson8:

- type="transcript", is_final=false -- update interim display (not saved)
- type="transcript", is_final=true -- queueConsultationSegment() -> POST /transcript
- type="flush_confirmation" -- resolves the end-of-session flush promise

Session shutdown flow: 1. capture.flush() -- disconnects audio source, posts "flush" to worklet, waits up to 2s. 2. Sends { type: "flush_request", id: <uuid> } to Reson8 over WebSocket. 3. Waits up to 12s for flush_confirmation. 4. Awaits consultation.queue to confirm all pending saves. 5. Calls closeConsultationCapture().

Save recovery: segments that fail POST /transcript remain in consultation.pending. The UI locks all other actions and shows "Retry saving transcript" until pending is empty.

8.2 Caregiver voice (app.js)

Uses Reson8 Turns (request/response, not streaming). A separate short-lived token is requested for each question. The transcript is inserted into the chat input and submitted automatically when the turn ends.

9. Frontend Architecture

9.1 App shell

The UI is a single-page app with three tabs rendered inside a persistent shell:

```
.app-shell
  |-- .sidebar          - dark navigation rail (250 px, fixed)
  |     `-- .nav-items  - Doctor / Caregiver / Report
  `-- .main-shell       - scrollable content area
        `-- active tab content
```

CSS custom properties define the design tokens:

```
--ink: #0f1923;     --surface: #ffffff;   --surface-raised: #f4f6f8;
--navy: #132f4c;    --violet: #6b6ef9;    --mint: #14b8a6;
--amber: #f59e0b;   --error: #dc2626;
```

9.2 Patient identity rendering -- NEW

When a discharge document is uploaded, case.patientName is populated from the PDF. renderPatient() runs on every setCase() call and fills every [data-patient] slot:

```
function renderPatient() {
  const name = caseView.case.patientName;
  const first = name?.split(" ")[0];
  const text = {
    full: name ?? "Patient not yet identified",
    first: first ?? "the patient",
    possessive: first ? `${first}'s` : "Patient's",
    initial: (first ?? "P")[0].toUpperCase(),
  };
  document.querySelectorAll("[data-patient]").forEach((el) => {
    el.textContent = text[el.dataset.patient];
  });
}
```

HTML elements carry data-patient="full", data-patient="first", data-patient="possessive" or data-patient="initial" attributes. All are updated reactively whenever the case view changes. The report tab heading also shows the name:

```
`<h2>Central report${caseView.case.patientName
  ? ` - ${esc(caseView.case.patientName)}` : ""}</h2>`
```

9.3 Doctor tab -- three-stage workflow strip

```
01 LIVE CONSULTATION  ->  02 SOURCE MATERIAL  ->  03 AI-ASSISTED REVIEW
```

Stage 01 (consultation):

- Consent checkbox + language selector
- Start / End consultation buttons
- Interim transcript display

- Speaker diarization mapping (Voice 1 / Voice 2 -> clinician / patient)
- Manual transcript fallback (single line entry)
- PDF transcript import
- Clear transcript button

Stage 02 (source material):

- PDF upload for discharge document

Stage 03 (AI review):

- Generate draft -> displays summary + care tasks
- Inline edit of summary text and task list
- Source chips on each task (hover to see passage text)
- Missing information warnings
- Approve button (with approvedBy capture)

9.4 Caregiver tab

- At-home view with active care task checklist
- Voice input prompt (Reson8 Turns)
- Conversation list with source chips (answer), pending confirmation cards (update), and unresolved badges
- Task status badges: open / in-progress / done / blocked

9.5 Report tab

- Approved summary
- Full task list with latest status and progress event notes
- Patient name in heading when available
- Unresolved questions list
- Conversation history

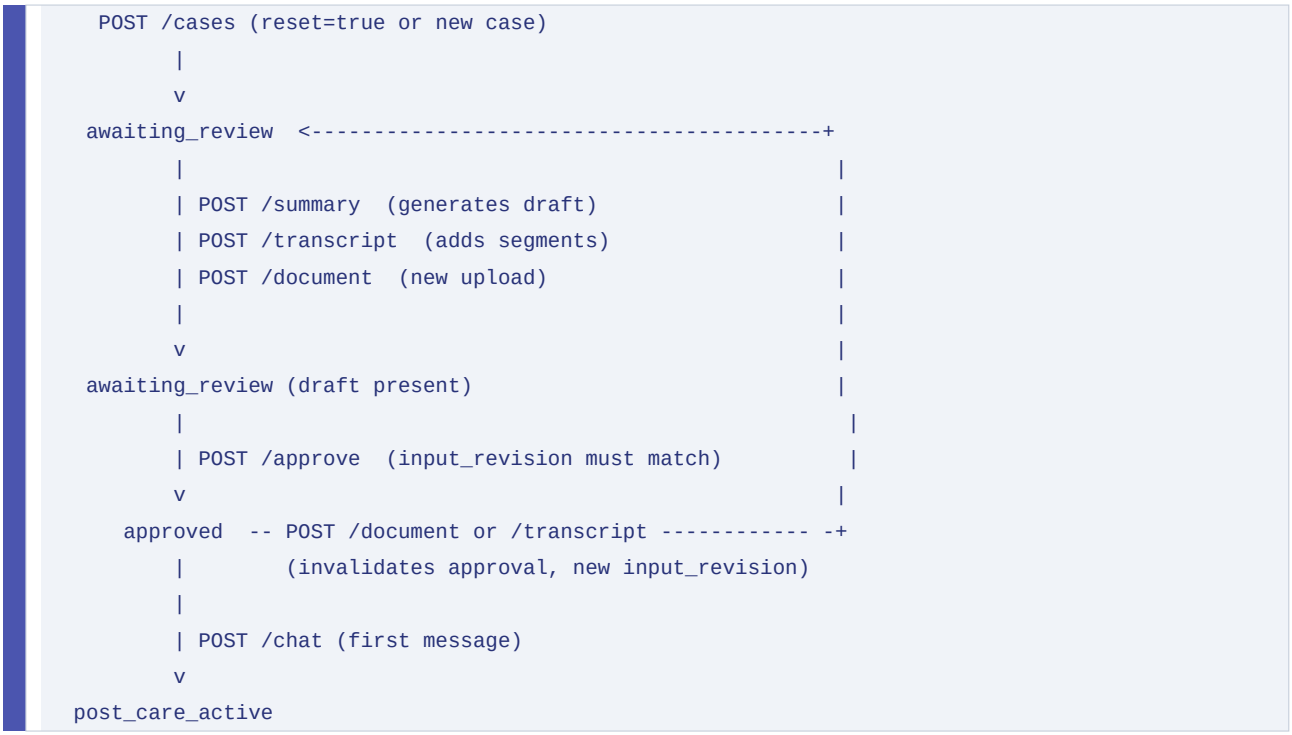
9.6 State management (app.js)

All state lives in a single caseView variable. Every API response that returns a CaseView calls setCase(view) which updates the variable and re-renders all three tabs. The UI derives its enabled/disabled state entirely from the server response.

Key helpers:

- api(path, opts) -- fetch wrapper; parses JSON, throws on non-2xx.
- run(target, fn) -- async action wrapper; disables target button, catches errors.
- esc(str) -- HTML-escapes user-supplied strings before inserting into innerHTML.

10. Workflow State Machine



An approved case can transition back to awaiting_review by uploading new sources, but the previous SummaryVersion is preserved. Post-care questions are gated on post_care_ready() which checks no sources have changed since the approval.

11. Demo Fixtures

After the latest pull the fixtures directory contains:

File	Purpose
Anna_Mueller_Discharge_Report.pdf	Sample discharge summary for upload in Stage 02
Anna Mueller_Consultation_Transcript.	Sample transcript for PDF import in Stage 01

Upload the discharge report first -- "Anna Mueller" is extracted automatically and displayed across the UI. Import the transcript PDF to populate the consultation source. Both files use entirely fictional patient data.

12. Configuration

All configuration is via environment variables (.env file):

Variable	Default	Purpose
ANTHROPIC_API_KEY	required	Backend-only.
ANTHROPIC_MODEL	claude-opus-5-5	Model for all Claude calls.
ANTHROPIC_EFFORT	omitted	low/medium/high/xhigh/max budget tokens.
ANTHROPIC_FALLBACKS	omitted	default enables server-side refusal fallback.
RESON8_API_KEY	required	Exchanged for short-lived browser tokens.
PORT	3001	uvicorn listen port.

13. Testing

```
uv run pytest                                    # Python workflow tests
node --test tests/consultation_frontend.test.cjs     # simulated JS speech-controller tests
```

Python tests use a scripted fake LLM client -- no API keys required. They cover the full HTTP -> storage -> AI boundary with deterministic fake responses.

The new tests/test_transcript_import.py covers the header block detection feature:

- Metadata-only labels (Date, Purpose, etc.) are not treated as speakers.
- Participant lines inside a header block (Patient: Anna Mueller) are skipped.
- Stray metadata fields mid-transcript are filtered out.

JS tests use fake WebSocket and getUserMedia APIs to simulate multi-speaker session persistence, finalization timing, and save-recovery flows.

14. Security Notes

- Permanent API keys (ANTHROPIC_API_KEY, RESON8_API_KEY) never leave the backend process.
- All user-supplied strings rendered into innerHTML are passed through esc().
- PDF uploads are validated by magic bytes (%PDF) and capped at 10 MB.
- Transcript segment IDs are validated by regex before persistence.
- The demo is single-tenant (one fixed case ID "demo-patient"). There is no authentication.
- The app does not store raw audio. Only finalized transcript text is persisted.